Reporte

Nombres: Equipo 11 Christopher Cortez Garcia Oscar Elias Vera Moreno Christofer Aaron Torres Puga	**Matrículas**: 2159313 2160859 2159639
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural del sistema (Fitzone)
Fecha: 17/10/2026	
Bibliografía (Formato APA) Documento de caso asignado enviado por el profesor. (2026). Caso 11: FitZone.	

Objetivo: Documentar cómo se usó la inteligencia artificial como copiloto en el diseño arquitectural de FitZone: qué se pidió, qué respondió, qué errores o supuestos se detectaron y qué decidió el equipo. La IA propone y revisa; las decisiones y su justificación son del equipo, conforme a los criterios de integridad académica de la guía.

Procedimiento:

Se conservaron los prompts enviados a la IA, se registraron las iteraciones, se verificó cada resultado contra el BRD y las User Stories, y se anotaron las correcciones. Las verificaciones técnicas (validez del documento, sintaxis de los diagramas y pruebas automáticas) se registran en la sección 4.

Herramienta y contexto

Aspecto	Detalle
Herramienta	Claude (Anthropic), en una sesión de trabajo con el BRD, las User Stories y la guía oficial de la Unidad 3 adjuntos.
Insumos entregados a la IA	BRD y User Stories de FitZone (U2), guía oficial de la Unidad 3 (Actividad_Unidad_3_CLN_ArquitecturaAlumnos.pdf).
Datos excluidos	No se envió información ajena al caso académico de FitZone.
Salidas de la IA	Informe de arquitectura (.docx, 13 páginas), prompts/XML de draw.io para el diagrama de clases y los dos diagramas de secuencia, código base en Python (dominio, aplicación, infraestructura).
Responsabilidad	El equipo revisó el informe y lo aprobó con observaciones de formato (reducción de preguntas de entrevista, incorporación de ideas del borrador de un compañero). Cada integrante debe poder defender las decisiones sin leer una respuesta generada.

Ciclo de trabajo aplicado

Paso	Acción	Evidencia
1	Preparar contexto	Se seleccionaron fragmentos relevantes del BRD, las User Stories, el BPMN y el glosario oficiales del Equipo 11.
2	Solicitar propuesta	Se pidió una alternativa concreta a la IA, exigiendo que declarara sus supuestos.
3	Comparar	Se contrastó la respuesta contra el caso FitZone y los conceptos DDD de la unidad.
4	Corregir	Se modificaron nombres, responsabilidades, límites y dependencias que no correspondían.
5	Validar	Se revisó trazabilidad, consistencia entre diagramas y comportamiento esperado (Etapa 9).
6	Registrar	Se guardó prompt, respuesta, decisión humana y versión resultante en la bitácora siguiente.

Bitácora de iteraciones

Las iteraciones 1 a 4 corresponden a la bitácora oficial incluida en el informe de arquitectura (Etapa 8). Las iteraciones 5 y 6 documentan trabajo posterior con IA, ya con el informe validado: la corrección de los diagramas UML en draw.io y la generación del código base (entregable 06).

Reporte

It.	Objetivo	Prompt (resumen)	Respuesta útil	Error o supuesto detectado	Decisión del equipo
1	Elegir el patrón de lógica de negocio	Comparar Transaction Script, Table Module y Domain Model para FitZone con base en las reglas del BRD de U2, exigiendo declarar supuestos.	Tabla comparativa de 4 criterios + justificación por alternativa.	La IA inicialmente sugería una combinación de patrones sin delimitar dónde se usaría cada uno.	Se adoptó Domain Model puro (ADR-01), sin combinación de patrones.
2	Construir el mapa de dominio	Proponer candidatos a Entidades, Value Objects, Aggregates y Domain Events a partir del glosario y las reglas de U2, indicando dudas para validación humana.	Mapa de dominio con 2 Aggregates (ClaseProgramada, Socio) y su diagrama visual.	La IA proponía inicialmente Reserva como Aggregate independiente, ignorando el riesgo de condiciones de carrera.	Se descartó Reserva como Aggregate independiente (Etapa 3.3).
3	Diseñar la Service Layer	Proponer firmas de Application Services a partir de los 5 casos de uso, separando coordinación de reglas de dominio.	Tabla de 5 servicios/métodos + checklist de revisión obligatoria respondido.	Ninguno relevante; se verificó que ningún método contuviera lógica de negocio directamente.	Se aceptaron los 5 métodos propuestos, ajustando solo nombres al glosario del equipo.
4	Revisar el diseño con SOLID	Señalar posibles violaciones SOLID en la Service Layer, Repository y Aggregates, sin reescribir todo el diseño.	5 hallazgos con evidencia, impacto y cambio sugerido por principio.	La IA sugería heredar ReservaDropIn de Reserva; riesgo de violar Liskov detectado por el equipo.	Se usó un atributo esDropIn en vez de herencia (hallazgo 3, Etapa 7).
5	Generar los diagramas UML en draw.io	Prompt de texto natural para el diagrama de clases y los 2 diagramas de secuencia, con instrucciones explícitas para evitar un layout tipo BPMN.	3 prompts de draw.io en lenguaje natural, uno por diagrama.	El generador de IA de draw.io producía texto encimado dentro de las cajas y un layout tipo BPMN, incluso tras instrucciones más explícitas.	Se reemplazaron los prompts en lenguaje natural por XML exacto de draw.io (mxGraph), generado y validado con un script, eliminando el problema de encimado.
6	Generar el código base (entregable 06)	Pedir una implementación boilerplate en Python (dominio, aplicación, infraestructura) a partir del informe de arquitectura ya validado.	Paquete con 2 Aggregates, 5 Application Services, repositorios en memoria, Unit of Work y 9 pruebas automáticas.	El límite de reservas simultáneas por plan se calculaba contra el total de reservas de la clase, en vez de contra las reservas del propio socio.	Se corrigió el cálculo para contar las reservas activas del socio a través de todas las clases, antes de integrar el código al entregable.

Verificaciones técnicas realizadas

Verificación	Cómo se hizo	Resultado
Validez del .docx	Conversión a PDF (LibreOffice) y revisión visual página por página antes de entregar.	Validación aprobada, sin errores de formato.
Sintaxis de los diagramas	Validación del XML de draw.io (mxGraph) con un parser antes de entregarlo.	Sin errores; diagramas renderizados sin texto encimado.
Consistencia entre diagramas	Comparación de los participantes y métodos de los 2 diagramas de secuencia contra las clases del diagrama de clases.	Todos los nombres coinciden.
Comportamiento esperado	9 pruebas automáticas (pytest) sobre los 5 casos de uso del código base.	9 de 9 aprobadas.

Validación humana de los aportes de la IA

Aporte de la IA	Contraste con la Unidad 2	Resultado
Domain Model como patrón	Las reglas del BRD (N1-N4) y US-001/US-013 exigen proteger invariantes de cupo y vigencia.	Aceptado (ADR-01)
Reserva como Aggregate independiente	Contradice el límite de consistencia necesario para evitar condiciones de carrera al reservar.	Rechazado
Herencia de ReservaDropIn sobre Reserva	Rompería el contrato esperado de Reserva en ControlAccesoService y ReservaService (riesgo Liskov).	Rechazado — se usó un atributo esDropIn
Umbral de no-shows fijo dentro de Socio	Cada cambio de política de sanciones obligaría a modificar la entidad (riesgo Open/Closed).	Corregido — se aisló como parámetro configurable
Repositorio único para Socio y ClaseProgramada	ControlAccesoService dependería de métodos de ClaseProgramada que nunca usa (violación de I).	Rechazado — se separaron ISocioRepository e IClaseProgramadaRepository
Límite de reservas simultáneas contado por clase	No corresponde al negocio: el límite es por socio (US-001), no por clase.	Corregido en el código base antes de integrarlo

Errores detectados y correcciones

- **Diagrama de clases:** El generador de IA de draw.io producía un diagrama de clases con texto encimado y layout tipo BPMN en los dos primeros intentos.
- **Código base:** El código base calculaba el límite de reservas simultáneas por plan contra el total de la clase, no contra las reservas del propio socio.
- **Formato de diagramas:** La primera versión de los diagramas usaba PlantUML; se corrigió a draw.io por preferencia explícita del equipo.

Propuestas de la IA rechazadas o corregidas (pregunta de defensa)

Reserva como Aggregate independiente, un Aggregate financiero de "saldos a favor" para reembolsos, herencia de ReservaDropIn sobre Reserva, y los dos primeros diagramas de clases generados automáticamente por el asistente de IA de draw.io sin control de layout.

Prompts utilizados

7.1 Mensaje 1 (enviado por el equipo)

Realiza un boceto de la siguiente actividad, en base a la actividad anterior, donde analices detalladamente cada uno de los apartados para crear una versión decente que posteriormente analizaremos, puedes utilizar cualquier formato, al fin de cuentas tu aporte es solo un boceto de lo que será la actividad completa que nosotros crearemos.

[Adjuntando: Todos los documentos de la Actividad 2]

[Adjuntando: Actividad_Unidad_3_CLN_ArquitecturaAlumnos.pdf]

Formatos de prompt de la guía aplicados al caso (reproducibles)

Son los formatos de prompt sugeridos por la guía oficial, completados con los datos de FitZone. Permiten que el equipo los reproduzca y compare resultados.

Tema	Prompt
Patrones	Con base en las reglas de cut-off, FIFO y bloqueo del BRD de FitZone (N1-N4) y los casos de uso CU-01 a CU-05, compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
Dominio	Revisa este glosario y estas reglas de FitZone [Socio, Membresía, ClaseProgramada, Reserva, ListaDeEspera, Instructor]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
Service Layer	A partir de los casos de uso CU-01 a CU-05 de FitZone, propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
SOLID	Analiza este diseño de FitZone [Service Layer, Repository, Aggregates] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
Diagramas	Convierte esta arquitectura validada de FitZone en un diagrama de draw.io (prompt de texto o XML mxGraph). No agregues clases, reglas o relaciones que no aparezcan en la descripción.

Declaración de uso responsable y pendientes

- Cada aporte de la IA fue revisado contra el BRD y las User Stories antes de integrarse.
- Los supuestos y decisiones dudosas quedaron documentados en la sección 3.3 del informe ("Decisiones dudosas o descartadas").
- Pendiente del equipo: subir el código base y los archivos fuente de los diagramas (.drawio) al repositorio de GitHub.

- Pendiente del equipo: cada integrante debe ensayar la defensa de las decisiones sin apoyo de la IA.